

DBMS LAB – TASK 6 – CSE 3

SQL Subqueries / Nested Queries using the Taxation Database

1. Aim

To understand and apply **SQL Subqueries (Nested Queries)** to retrieve information based on the result of another SQL query.

2. Objective

By the end of this laboratory experiment, students should be able to:

- Understand the concept of subqueries.
- Use single-row subqueries with comparison operators.
- Use multi-row subqueries with **IN**, **NOT IN**, **ANY**, and **ALL**.
- Use aggregate functions inside subqueries.
- Combine subqueries with **WHERE** conditions.
- Solve real-world taxation queries using nested queries.

3. Concepts Covered

- Subqueries / Nested Queries
- Single-row Subqueries
- Multi-row Subqueries
- **IN** and **NOT IN**
- **ANY** and **ALL**
- Aggregate Functions with Subqueries
- Subqueries with **WHERE**
- Subqueries with **FROM**
- Nested Query Execution

4. Prerequisites

Students should already know:

- `SELECT, FROM, WHERE`
- Comparison Operators
- Logical Operators
- Aggregate Functions
- `GROUP BY`
- `HAVING`
- `ORDER BY`
- `INNER JOIN, LEFT JOIN, RIGHT JOIN`
- Table aliases

These concepts were covered in the previous laboratories. Task 5 specifically established aggregate functions and indicated their use inside upcoming subqueries.

5. Database to be Used

Continue using the existing database:

`USE taxation_db;`

Existing tables:

- `Taxpayer`
- `Income_Record`
- `Income_Category`
- `Financial_Year`

No new tables are required.

Insert if any additional data required as per Queries

6. Introduction to Subqueries

A **subquery** is a query written inside another SQL query.

The inner query executes first, and its result is used by the outer query.

General Syntax

```
SELECT column_name  
FROM table_name  
WHERE column_name operator  
      (SELECT column_name  
        FROM table_name  
        WHERE condition);
```

Example

Display taxpayers whose income is greater than the average income:

```
SELECT taxpayer_id, income_amount  
FROM Income_Record  
WHERE income_amount >  
      (SELECT AVG(income_amount)  
        FROM Income_Record);
```

Here:

- Inner query → calculates average income.
 - Outer query → finds records greater than that average.
-

7. Single-Row Subquery

A **single-row subquery** returns only one value.

It can generally be used with operators such as:

```
=  
>  
<  
>=  
<=  
<>
```

Example

Display the income record having the highest income:

```
SELECT *  
FROM Income_Record  
WHERE income_amount =  
      (SELECT MAX(income_amount)  
       FROM Income_Record);
```

8. Multi-Row Subquery

A **multi-row subquery** returns multiple values.

Operators such as:

IN
NOT IN
ANY
ALL

can be used with multi-row subqueries.

Example – IN

Display taxpayers who have Business income:

```
SELECT taxpayer_id  
FROM Income_Record  
WHERE category_id IN  
      (SELECT category_id  
       FROM Income_Category  
       WHERE category_name = 'Business');
```

9. Subquery with Aggregate Functions

Aggregate functions learned in Task 5 can be used inside subqueries.

Examples:

AVG()
MAX()
MIN()
SUM()
COUNT()

Example

Display income records greater than the average income:

```
SELECT *  
FROM Income_Record  
WHERE income_amount >  
      (SELECT AVG(income_amount)  
       FROM Income_Record);
```

This creates the following flow:

Income Records
↓
AVG(income_amount)
↓
Average Income
↓
Outer Query
↓
Records above Average

10. Subquery with IN

IN checks whether a value exists in the result returned by a subquery.

Syntax

```
SELECT column_name
FROM table_name
WHERE column_name IN
    (SELECT column_name
     FROM table_name
     WHERE condition);
```

Example

Display income records belonging to taxpayers who are Business Owners:

```
SELECT *
FROM Income_Record
WHERE taxpayer_id IN
    (SELECT taxpayer_id
     FROM Taxpayer
     WHERE occupation = 'Business Owner');
```

11. Subquery with NOT IN

NOT IN returns records whose value does not belong to the result of the subquery.

Example

Display taxpayers who have no income records in the Investment category:

```
SELECT *
FROM Taxpayer
WHERE taxpayer_id NOT IN
    (SELECT taxpayer_id
     FROM Income_Record
     WHERE category_id = 3);
```

12. Subquery with ANY and ALL

ANY

The condition becomes true if it satisfies the comparison with **at least one value** returned by the subquery.

Example:

```
SELECT *  
FROM Income_Record  
WHERE income_amount > ANY  
      (SELECT income_amount  
       FROM Income_Record  
       WHERE category_id = 3);
```

ALL

The condition becomes true only if it satisfies the comparison with **all values** returned by the subquery.

Example:

```
SELECT *  
FROM Income_Record  
WHERE income_amount > ALL  
      (SELECT income_amount  
       FROM Income_Record  
       WHERE category_id = 3);
```

Note: **ANY** and **ALL** are introduced here as multi-row subquery operators. More complex set-comparison applications can be practiced in later experiments.

13. Laboratory Activities

Part A – Basic Subqueries

Level 1 – Understanding

Task 1: Display the income record having the highest income.

Hint: Use `MAX()` inside a subquery.

Task 2: Display the income record having the lowest income.

Hint: Use `MIN()` inside a subquery.

Task 3: Display income records having an income greater than the average income.

Hint: Use `AVG()` inside a subquery.

Task 4: Display income records having an income equal to the highest income recorded.

Hint: Use `MAX()` with `=`.

Task 5: Display taxpayers whose occupation is `Business Owner`.

Hint: Use a subquery with `IN`.

Level 2 – Application

Task 1: Display taxpayers who have at least one income record.

Task 2: Display taxpayers who have income in the `Business` category.

Task 3: Display income records belonging to the financial year `2025-2026`.

Hint: Use a subquery on `Financial_Year`.

Task 4: Display income records whose amount is greater than the minimum Business income.

Task 5: Display income records whose amount is less than the maximum Salary income.

Task 6: Display taxpayers who have income records greater than the average income.

Task 7: Display income categories that have at least one income record.

Task 8: Display taxpayers who have no income records in the Investment category.

Hint: Use `NOT IN`.

Level 3 – Medium to Advanced

Task 1: Display the taxpayer having the highest recorded income.

Hint: Use `MAX()` inside a subquery.

Task 2: Display all income records having an amount greater than the average Business income.

Task 3: Display taxpayers whose income is greater than the average income of all taxpayers.

Task 4: Display income records having an amount greater than **ANY** income recorded under the Investment category.

Task 5: Display income records having an amount greater than **ALL** income recorded under the Investment category.

Task 6: Display the income category that contains the highest income record.

Hint: Use a subquery with `MAX()`.

Task 7: Display the financial year having the highest total income.

Hint: Use a subquery with `SUM()` and `GROUP BY`.

Task 8: Display the taxpayer(s) whose total recorded income is greater than the average total income of taxpayers.

14. Real-World Taxation Analysis

Task 1

Identify the taxpayer who has the highest individual income.

Hint: Use `MAX()` with a subquery.

Task 2

Identify taxpayers whose income is above the overall average income.

Hint: Use `AVG()` with a subquery.

Task 3

Identify the income category containing the highest income record.

Hint: Use `MAX()` and a nested query.

Task 4

Identify taxpayers who have Business income but no Investment income.

Hint: Use `IN` and `NOT IN`.

Task 5

Identify income records whose amount is greater than every Investment income record.

Hint: Use `ALL`.

Task 6

Identify income records whose amount is greater than at least one Investment income record.

Hint: Use `ANY`.

Task 7

Display the taxpayer(s) having the highest total income.

Hint: Use `SUM()`, `GROUP BY`, and a subquery.

Task 8

Generate a list of income records whose amount is above the average income of their corresponding category.

Hint: This introduces the idea of comparing a record with a calculated group value; **do not use a correlated subquery yet.**

15. Important Concept

Students should understand the progression:

Individual Records



Aggregate Functions



GROUP BY



HAVING



ORDER BY



Subquery



Single-Row Subquery



Multi-Row Subquery



IN / NOT IN



ANY / ALL



Nested Analysis

The key idea is:

Outer Query → asks the final question

Inner Query → provides the value(s) required to answer it

16. Task 6 Submission

Students should create:

StudentID-repo

└─ Week-6-Lab

└─ week6_subqueries.sql